Question 1. [5 points] What output is printed by the following code (which begins on the left and continues on the right)?

```
public class Q4 {
  public static void main(
      String[] args) {
    List<Integer> list =
      Arrays.asList(1, 2, 3, 4, 5);

    Iterator<Integer> i =
      list.iterator();
    System.out.println(i.next());
    System.out.println(i.next());
    if (i.hasNext()) {
      System.out.println(i.next());
    }
    if (i.hasNext()) {
      System.out.println(i.next());
    }
  }
}
```

Answer:

1
2
3
4

This Java program uses an Iterator to go through a list of numbers one at a time.

The output is 1, 2, 3, and 4, each on a separate line. This is because the iterator goes through the list one number at a time using `next()`.

The `hasNext()` method checks if there is another number available before printing it.

The number 5 is not printed because the program only calls `next()` four times.

Question 2. [7 points] Consider the following code:

```
public class Foo {
  public static void main(String[] args) {
    List<Integer> list = Arrays.asList(5, 6, 4, 3, 8, 1);
    int result = 1;

    Iterator<Integer> i = list.iterator();
    while(i.hasNext()) {
      if ((i.next())%2 == 0) {
         result *= i.next();
      } else if (i.next()%2 != 0) {
         result += i.next();
      }
      System.out.println("Result: " + result);
    }
    System.out.println("Done");
  }
}
```

What output is printed when this code is executed? Briefly justify your answer. **Note:** `%2==0` determines if a value is *even* and `%2!=0` determines if a value is *odd*. Furthermore, the first print statement is *inside* the loop.

Answer:

Result: 1
Result: 3
Result: 3
Done

Same as Q1. This is because every time `i.next()` is called, the iterator moves to the next number.

First loop: `i.next()` gets 5 (odd), so the `if` is false. The `else if` gets 6 (even), so that is also false. Result stays 1.

Second loop: `i.next()` gets 4 (even), so the `if` is true. The next `i.next()` gets 3, so `result = 1 × 3 = 3`.

Third loop: `i.next()` gets 8 (even), so the `if` is true. The next `i.next()` gets 1, so `result = 3 × 1 = 3`.

End: No more numbers remain, so the loop stops and prints `Done`.

Question 3. [5 points] Consider the following code:

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);

Iterator<Integer> i = list.iterator();
while (i.hasNext()) {
    System.out.println(i.next());
    System.out.println(i.next());
}
System.out.println("done");
```

The program prints 1, 2, 3, 4, and 5, each on a separate line, and then throws a `NoSuchElementException`.

This happens because the loop calls `i.next()` twice but only checks `i.hasNext()` once.

When it reaches the last number (5), the second `next()` tries to access a number that doesn't exist. Therefore, `"done"` is never printed.

What output is printed when this code is executed? Explain your answer briefly.

Answer:

1
2
3
4
5
Cause Exception (NoSuchElementException)

Question 4. [5 points] Consider the following code—the `Box` class on the left, and a code fragment using it on the right:

```
public class Box
    implements Comparable<Box> {
  public int x;

  public Box(int val) { x = val; }

  public int compareTo(Box m) {
    return m.x - this.x;
  }
}
```

```
Box[] boxes = {
    new Box(3),
    new Box(1),
    new Box(2)
};

Arrays.sort(boxes);
for (Box b : boxes) {
    System.out.println(b.x);
}
```

What output is printed by the code on the right?

Answer:

3
2
1

This is because the `compareTo()` method returns `m.x - this.x`, which sorts the Box objects in descending order (largest to smallest).

The `Arrays.sort()` method uses `compareTo()` to arrange the objects, and the loop prints their values.

Question 5. [5 points] Consider the following code—the `Frob` class on the left, and a code fragment using it on the right:

```
public class Frob
    implements Comparator<String> {

  public int compare(
      String lhs, String rhs) {
    return lhs.length() -
           rhs.length();
  }

}
```

```
String[] names =
    { "Alice", "Bob", "Carl" };

Frob f = new Frob();
System.out.println(
    f.compare("Alice", "Bob"));

Arrays.sort(names);
for (String n : names) {
    System.out.println(n);
}
```

What output is printed by the code on the right?

Answer:

```
2
Alice
Bob
Carl
```

The first statement prints 2 because `"Alice"` has 5 letters and `"Bob"` has 3 letters, so $5 - 3 = 2$.

The `Arrays.sort(names)` method sorts the names alphabetically because the custom comparator `f` is not passed into the sort method.

Since the names are already in alphabetical order, they remain unchanged.

Question 6. [5 points] Consider the following method, which is intended to return the sum of the integers in a collection of integer values:

```
public static int sum(Collection<Integer> coll) {
    int sum = 0;
    for (int i = 0; i < coll.size(); i++) {
        sum += coll.get(i);
    }
    return sum;
}
```

Briefly explain the error in this method and how to fix it. (Hint: consider the parameter's data type.)

The error is that `Collection<Integer>` does not have a `get(i)` method.

The `get(i)` method works with a `List`, such as an `ArrayList`, but not with a general `Collection`.

Not every collection allows you to access elements by their index.

Remember what we said in class? One reason to use iterator is that an iterator allows us to get elements in a collection without knowing how the collection is implemented.

Question 7. [5 points] Complete the following generic method. It should return the minimum of the two parameter values. (Hint: note that the type parameter `E` is guaranteed to implement `Comparable`.)

```
public static<E extends Comparable<E>> E min(E val1, E val2) {



}
```

```
if (val1.compareTo(val2) < 0){
    return val1;
}else{
    return val2;
}
```

This question is to check if you know there is compareTo() method in Comparable interface and how to use it.

Remember for your exam:

- `compareTo()` `<` `0` → smaller
- `compareTo()` `==` `0` → equal
- `compareTo()` `>` `0` → greater

Programming Questions

To get started, use a web browser to download the zipfile as specified by your instructor. Import it as an Eclipse project using File → Import... → General → Existing Projects into Workspace → Archive file.

Important: You may use the following resources:

- The textbook
- The lecture notes posted on the course web page
- Your previous labs and assignments

Do not open any other files, web pages, etc.

Question 8. [25 points] Complete the implementation of the `Pokemon` class.

(a) Fields, Constructors, and Getters (12 points)

The class should have three fields:

1. A primary type, whose data type is `PokemonType`
2. A secondary type, whose data type is `PokemonType`
3. A name, whose data type is `String`

There are two constructors:

- The first takes primary type and name as parameters, storing the parameter values in the corresponding fields, and sets the object's secondary type to `PokemonType.NONE`
- The second takes primary type, secondary type, and name, storing the parameter values in the corresponding fields

There should be getters `getPrimaryType`, `getSecondaryType`, and `getName` which return the value of the appropriate field.

(b) Comparable Interface (13 points)

The class should implement the `Comparable<Pokemon>` interface. The `compareTo` method should compare first by primary type, then by secondary type (if the primary types are equal), and finally by name (if the primary and secondary types are equal).

Note that because `PokemonType` is an enumeration, it implicitly implements `Comparable`, so you can compare `PokemonType` values using the `compareTo` method (as you did with `Suit` and `Rank` values in Labs 10 and 11.)

There are unit tests in the `Q11Test` class: make sure they pass.

Question 9. [25 points] In the `Q12` class, implement the `isSorted` static method. The method takes a `List` of elements of type `E`, and returns `true` if the list is sorted. Note that `E` is guaranteed to implement `Comparable<E>`, so you can call `compareTo` on values of type `E`.

(a) Empty and Single-Element Lists (5 points)

Ensure that the `isSorted` method returns `true` if the list is empty or has exactly one element.

(b) Checking Whether a List Is Sorted (15 points)

Implement the logic to determine whether the list is sorted. A list is sorted if and only if for each element other than the first, the previous element is not greater.

Your method should use `compareTo` to compare elements and return `true` if the list is sorted and `false` otherwise. Note that a sorted list may contain duplicate elements.

(c) Time Complexity (5 points)

To receive full credit for your solution, your method must complete in $O(N)$ time, where N is the number of elements in the list.

However, you can expect to receive 80% of full credit if your algorithm is correct but completes in $O(N^2)$ time. (A slow but correct algorithm will get a much better score than an incorrect algorithm.)

There are JUnit tests in the `Q12Test` class. Make sure they pass.